

POLE+O：用五种基础类型破掉建知识图谱最难的那一步

POLE+O: The 5-Type Ontology

核心内容

Q：这篇文章要解决的是什么问题？ A：空白画布问题。每个知识图谱项目开头都一样：有人打开白板问「那么，我们的实体有哪些？」，接着是好几天的争论——「会议」算节点还是关系？「地点」值得单独一个类型还是只是个属性？作者的判断是：卡住的原因不是缺领域知识，而是**建模一个领域有太多种都说得通的方式，而没有一个起点作锚**。每个选项看起来都合理，没有哪个明显是错的，于是争论绕圈。

Q：POLE+O 是什么？ A：五个基础实体类型——Person（任何人类实体）、Organization（群体与公司）、Location（物理或逻辑的地点）、Event（发生的事）、Object（其余一切）。名字来自执法与情报分析领域，POLE 四个字母在那里用了几十年，作用是把散在各个机构的数据连起来。「+O」加上 Object 兜住剩下的世界。你的实体不属于前四种，那它就是 Object。

Q：这五种类型怎么和具体领域接上？ A：靠 Neo4j 的多标签机制叠上去，而不是替换。医疗领域写成 `:Person:Patient`、`:Event:Diagnosis`、`:Object:Prescription`；软件工程领域写成 `:Person:Developer`、`:Event:Sprint`、`:Object:Issue`。**基础类型永远在，领域类型加具体性**。作者说这带来两样扁平模式给不了的东西。

Q：那两样东西是什么？ A：第一，可以在基础层查——一条 `MATCH (e:Event) WHERE e.date > date() - duration('P30D')` 就取回过去三十天里所有事件，不管它来自哪个领域（冲刺、诊断、事故、交易）。第二，**多源数据在基础层自动统一**：HR 系统里的 `:Person` 和项目跟踪系统里的 `:Person` 已经在同一个桶里，按邮箱或 ID 对齐就是一次合并，而不是一次迁移。

Q：作者凭什么说这比从零开始好？ A：他不主张 POLE+O 能回答所有建模问题，只主张它回答了第一个问题：「我的顶层类别有哪些」。而这个问题贵得出人意料——团队会在上面花掉好几天，因为**没有错的答案，只有取舍**。他的原话是：POLE+O 跳过这场争论，不是因为它完美，而是因为它**够用来开始**。而开始这件事本身重要：「一个第一周就能查的知识图谱，比一个你争论了一个月的模式，教给你更多关于这个领域的东西。」

Q：作者自己承认它在哪里不适用？ A：三处，说得很诚实。**抽象或概念性的领域**（金融工具、业务规则、政策、数学模型）映射不干净，Object 会变成一个万能兜底，而**「一个装了所有东西的兜底桶等于什么都没装」**。**层级很深的领域**（实体需要四层以上类型继承）会觉得这五种类型太薄，真正的建模工作还是全在领域层。已经在用 RDF/OWL 的环境（SNOMED、FIBO、schema.org），再叠一层 POLE+O 可能和已有的上层本体冲突。

背景补充 / 点评

作者 Konrad Kaliciński 是 Neo4j 的全栈开发者，文章原发在 Medium 的 Neo4j Developer Blog。起因是 Neo4j Labs 放出了一个叫 [Create Context Graph](#) 的命令行工具，用来脚手架式地生成带图记忆的 AI agent 应用。作者的判断是：这个项目里最值得复用的不是那个命令行工具，而是它底下那套本体。

几个背景值得补：

- POLE 的出处是情报分析，不是数据库领域。执法与情报机构用 Person / Organization / Location / Event 这四类做跨机构数据关联已经几十年。这是一套经过实战检验的上层本体（upper ontology）——一种不绑定具体领域、供各领域挂在下面的顶层分类。
- 文章后半段那套三层记忆架构（短期记忆存对话历史、长期记忆存实体关系、推理记忆存决策轨迹）也建在同一套本体上。推理记忆那部分是：agent 做决定时生成一个 `:DecisionTrace` 节点，连向若干 `:TraceStep`，每一步记下「想法、动作、观察」。
- [Neo4j Graph Data Modeling](#) 是官方免费课，讲图建模基础。

这篇文章最有价值的地方是它的诚实，不是它的方案。「Where POLE+O fits — and where it doesn't」那一节，作者主动说出「抽象领域映射不干净」「Object 会变成万能兜底」「和已有上层本体冲突」——一篇厂商博客愿意写这些，可信度高出一截。而那句「a catch-all that holds everything holds nothing」是全文最锐的一句。

留白的地方也有两处。第一，作者说「一旦你开始对真实数据写查询，模型会自己修正」——这句话很动人，但它成立的前提是改模式的代价低。在 Neo4j 里加个标签确实便宜，可在一个已经有下游消费者的系统里，改模式的代价完全不是这个量级，文章没提这个前提。第二，「POLE+O 作为 LLM 设计图模式时的锚」这个用法只用了一段，而这恰恰是当下最有实用价值的部分——把对话从「发明一套本体」变成「把这个 POLE+O 模型细化到我的领域」。

对我自己的启示，而且是反向的。我在做统一语义层，读到这篇的第一反应是「五种基础类型可以借」，但作者自己那节「哪里不适用」正好否掉了这个念头：

- 我的领域恰恰是他说的**那种抽象领域**。我的本体里核心概念是课程、考试、学员、知识点——这些倒是能落进 Person / Object / Event；但我真正在治理的东西是口径（什么叫「完成」、什么叫「参与」、及格线从哪儿读），那是业务规则，不是实体。按他的分类，业务规则会掉进 Object 那个兜底桶，而那等于没分类。
- 所以**我的语义层不是知识图谱形状的，而是「规则与算法的注册表」形状的**。这两件事都叫本体，但治理的对象不同：他治理的是实体分类，我治理的是口径定义。

真正能借的是另外两点，而且比五种类型有用得多：

1. 「**基础层永远在，领域层加具体性**」这个叠加结构。我的指标注册中心现在是一层平的——46 条指标各自独立。而实际上它们有共同的基础形态（人数类、比率类、明细类、分布类）。如果基础形态显式登记出来，「所有比率类指标的分母口径都要如实声明」这种横切约束就能在基础层上一次表达，而不是每条指标各写一遍。这正是我这两天在收的那种「同一件事写在多处」。

2. 「够用来开始比争论到完美更重要」这条，我要反向警惕。我的处境和他假设的处境相反——我已经有下游消费者（问答链路、演示站、评测），改口径的代价不低。他那句「模型会自己修正」在我这里不成立，而正因为不成立，我在前期多花时间把口径的正本放对，是有回报的，不是过度设计。这一点值得记住，免得下次看到「先跑起来再说」的论调时动摇。

文中金句

"The problem isn't a lack of domain knowledge. It's that there are too many valid ways to model a domain and no anchor to start from." 问题不在于缺少领域知识，而在于建模一个领域有太多种都说得通的方式，却没有一个起点作锚。

"Not because it's perfect, but because it's good enough to start." 不是因为它完美，而是因为它够用来开始。

"A knowledge graph you can query in week one teaches you more about your domain than a schema you've been debating for a month." 一个第一周就能查的知识图谱，比一个你争论了一个月的模式，教给你更多关于这个领域的东西。

"Object becomes a catch-all, and a catch-all that holds everything holds nothing." Object 变成了万能兜底，而一个装了所有东西的兜底桶，等于什么都没装。

全文翻译

空白画布问题

Every knowledge graph project starts the same way. Someone opens a whiteboard and asks: "So... what are our entities?" What follows is a multi-day debate about whether a "meeting" is a node or a relationship, whether "location" deserves its own type or is just a property, and whether the schema should mirror the source database or represent some idealized domain model.

每个知识图谱项目的开头都一样。有人打开白板，问：「那么……我们的实体有哪些？」接下来是持续好几天的争论：「会议」算节点还是关系？「地点」值不值得有自己的类型，还是只是个属性？模式应该照抄源数据库，还是表达某种理想化的领域模型？

The problem isn't a lack of domain knowledge. It's that there are too many valid ways to model a domain and no anchor to start from. Every option looks reasonable. None of them is obviously wrong. So the debate loops.

问题不在于缺少领域知识。问题在于建模一个领域有太多种都说得通的方式，却没有一个起点作锚。每个选项看起来都合理，没有哪个明显是错的，于是争论绕圈。

There's an anchor that solves this. It's been around for decades in intelligence analysis, and Neo4j just made it easy to use.

有一个锚能解决这件事。它在情报分析领域已经存在几十年，而 Neo4j 刚刚把它变得好用了。

POLE+O 到底是什么

Neo4j Labs recently released Create Context Graph, a CLI tool that scaffolds full-stack AI agent applications with graph-based memory. The tool is interesting, but the most reusable thing in the project isn't the CLI. It's the ontology underneath.

Neo4j Labs 最近放出了 Create Context Graph，一个命令行工具，用来脚手架式地生成带图记忆的全栈 AI agent 应用。工具本身有意思，但这个项目里最值得复用的东西不是那个命令行工具，而是它底下那套本体。

POLE+O defines five base entity types: Person (any human entity — patient, employee, developer); Organization (groups and companies — hospital, startup, team); Location (places, physical or logical — office, region, facility); Event (things that happen — sprint, transaction, incident); Object (everything else — document, product, file).

POLE+O 定义了五种基础实体类型：Person，任何人类实体，比如患者、员工、开发者；Organization，群体与公司，比如医院、初创公司、团队；Location，地点，物理的或逻辑的，比如办公室、区域、设施；Event，发生的事，比如冲刺、交易、事故；Object，其余一切，比如文档、产品、文件。

The name comes from law enforcement and intelligence analysis, where POLE (Person, Organization, Location, Event) has been used for decades to connect disparate data across agencies. The "+O" extends it with Object to cover the rest of the world. If your entity doesn't fit the first four, it's an Object.

这个名字来自执法与情报分析领域，POLE（Person、Organization、Location、Event）在那里被用了几十年，作用是把散落在各个机构的数据连起来。「+O」用 Object 把它扩展，兜住世界的其余部分。如果你的实体不属于前四种，那它就是 Object。

领域映射怎么落地

The power of POLE+O isn't the five types themselves. It's how domain-specific types layer on top using Neo4j's multi-label system.

POLE+O 的力量不在这五种类型本身，而在于领域专属的类型如何借 Neo4j 的多标签机制叠在它们之上。

In a healthcare domain: `CREATE (:Person:Patient {name: 'Jan Kowalski', dob: '1985-03-15'}); CREATE (:Event:Diagnosis {code: 'I25.1', date: '2026-05-10'}); CREATE (:Object:Prescription {drug: 'Aspirin', dosage: '100mg'})`

在医疗领域里这样写（见上方 Cypher）：患者是 `:Person:Patient`，诊断是 `:Event:Diagnosis`，处方是 `:Object:Prescription`。

In a software engineering domain: `CREATE (:Person:Developer {name: 'Alice'}); CREATE (:Organization:Team {name: 'Platform'}); CREATE (:Event:Sprint {number: 42}); CREATE (:Object:Issue {key: 'PLAT-1234'})`

在软件工程领域里则是：开发者是 `:Person:Developer`，团队是 `:Organization:Team`，冲刺是 `:Event:Sprint`，工单是 `:Object:Issue`。

Notice the pattern: `:Person:Patient`, `:Event:Sprint`, `:Object:Issue`. The base type is always there. The domain type adds specificity. This gives you two things you don't get with flat schemas.

注意这个模式：`:Person:Patient`、`:Event:Sprint`、`:Object:Issue`。基础类型永远在，领域类型负责加具体性。这带来两样扁平模式给不了的东西。

First, you can always query at the base level. This returns every event in the last 30 days — sprints, diagnoses, incidents, transactions — regardless of which domain they came from.

第一，你随时可以在基础层查。一条查询就取回过去三十天里的所有事件——冲刺、诊断、事故、交易——不管它们来自哪个领域。

Second, when you connect data from multiple sources, entities unify at the base level automatically. A `:Person` from your HR system and a `:Person` from your project tracker are already in the same bucket. Matching them by email or ID becomes a merge, not a migration.

第二，当你把多个来源的数据连起来时，实体在基础层自动统一。你 HR 系统里的 `:Person` 和项目跟踪系统里的 `:Person` 已经在同一个桶里了。按邮箱或 ID 把它们对上，是一次合并，而不是一次迁移。

为什么这比从零开始好

POLE+O doesn't answer every modeling question. But it answers the first one: "What are my top-level categories?" That question is deceptively expensive. Teams spend days on it because there's no wrong answer — only trade-offs.

POLE+O 回答不了每一个建模问题，但它回答了第一个：「我的顶层类别有哪些？」这个问题贵得出人意料。团队会在它上面花掉好几天，因为没有错的答案，只有取舍。

POLE+O skips the debate by giving you a classification that works across domains. Not because it's perfect, but because it's good enough to start.

POLE+O 给你一套跨领域都能用的分类，从而跳过这场争论。不是因为它完美，而是因为它够用来开始。

And starting matters. A knowledge graph you can query in week one teaches you more about your domain than a schema you've been debating for a month.

而开始这件事本身重要。一个第一周就能查的知识图谱，比一个你争论了一个月的模式，教给你更多关于这个领域的东西。

Once you start writing Cypher queries against real data, the model fixes itself. You notice that "meeting" shouldn't be a relationship — it should be an Event, because you need to attach participants, locations, and outcomes to it. The queries tell you. POLE+O gets you to that moment faster.

一旦你开始对真实数据写 Cypher 查询，模型会自己修正。你会发现「会议」不该是关系，它该是一个 Event，因为你需要往它上面挂参与者、地点和结果。是查询告诉你的。POLE+O 让你更快到达那个时刻。

哪里适用，哪里不适用

Good fit: Domains with concrete, real-world entities — healthcare, logistics, software engineering, legal. People, places, organizations, and things that happen map naturally. Multi-source integration: when you're merging data from different systems, the shared base types give you automatic unification points. Rapid prototyping: when you need a queryable graph in days, not weeks.

适用的场合：实体具体、贴近真实世界的领域——医疗、物流、软件工程、法律。人、地点、组织，以及发生的事，都能自然映射。多源集成：当你要把不同系统的数据合起来时，共享的基础类型给你自动的统一。快速原型：当你需要在几天而不是几周内拿到一个可查的图。

Less obvious fit: Abstract or conceptual domains — financial instruments, business rules, policies, mathematical models. These often don't map cleanly to Person/Organization/Location/Event. Object becomes a catch-all, and a catch-all that holds everything holds nothing.

不那么适用的场合：抽象或概念性的领域——金融工具、业务规则、政策、数学模型。这些东西往往映射不干净到 Person / Organization / Location / Event 上。Object 变成了万能兜底，而一个装了所有东西的兜底桶，等于什么都没装。

Deeply hierarchical domains: if your entities need 4+ levels of type inheritance, POLE+O's flat five-type layer may feel too thin. You'll end up doing most of the modeling work in the domain layer anyway.

层级很深的领域：如果你的实体需要四层以上的类型继承，POLE+O 那扁平的五种类型会显得太薄。真正的建模工作最后还是全落在领域层。

RDF/OWL environments: POLE+O is a property graph pattern. If your organization already uses formal ontologies (SNOMED, FIBO, schema.org), layering POLE+O on top adds a classification system that may conflict with existing upper ontologies.

RDF/OWL 环境：POLE+O 是一个属性图的模式。如果你的组织已经在用形式化本体（SNOMED、FIBO、schema.org），在上面再叠一层 POLE+O，等于加进一套可能和已有上层本体冲突的分类体系。

The honest take: POLE+O is a starting framework, not a complete ontology. It prevents the blank-canvas problem and gives you momentum. But the real modeling work — defining relationships, cardinality, temporal patterns — still happens after you pick your five buckets.

老实说：POLE+O 是一个起步框架，不是一套完整的本体。它避免了空白画布问题，给你推进的动能。但真正的建模工作——定义关系、基数、时间模式——仍然发生在你挑好那五个桶之后。

If you're using LLMs to help design graph schemas, POLE+O works well as an anchor. It gives both you and the model a framework instead of a blank page. The conversations shift from "invent an ontology" to "refine this POLE+O model for my domain."

如果你在用大模型帮忙设计图模式，POLE+O 当锚很好用。它给你和模型都提供一个框架，而不是一张白纸。对话于是从「发明一套本体」变成「把这个 POLE+O 模型细化到我的领域」。

三种记忆，建在同一套本体上

Create Context Graph doesn't just use POLE+O for static data. It builds a three-layer memory architecture on top of it: Short-term memory — conversation history, current session context; Long-term memory — persistent entity relationships modeled as a POLE+O knowledge graph; Reasoning memory — decision traces that capture why an agent did what it did.

Create Context Graph 不只把 POLE+O 用在静态数据上，它上面建了一套三层记忆架构：短期记忆，存对话历史与当前会话上下文；长期记忆，存持久的实体关系，建模成一张 POLE+O 知识图谱；推理记忆，存决策轨迹，记录 agent 为什么做了它做的那件事。

The reasoning memory is worth looking at. When an agent makes a decision, it creates a :DecisionTrace node linked to :TraceStep nodes — each recording a thought, an action, and an observation.

推理记忆那一层值得看。当 agent 做出一个决定时，它创建一个 `:DecisionTrace` 节点，连向若干 `:TraceStep` 节点，每一个记录一次想法、一次动作、一次观察。

You can't traverse a reasoning chain in a vector store. You can in a graph. "Vector stores give you recall, graphs give you understanding" is a line from the Neo4j team, and this architecture

makes it concrete.

在向量库里你没法遍历一条推理链，在图里可以。「向量库给你召回，图给你理解」是 Neo4j 团队的一句话，而这套架构把它落成了具体的东西。

自己上手试

If you want to experiment with POLE+O without installing anything, grab a Neo4j Aura free instance and create a small graph manually. Then map your own domain. Take 10 entities from your current project and classify each one as P, O, L, E, or O. Add domain labels. Start querying. The model will evolve — and that's the whole point.

如果你想不安装任何东西就试试 POLE+O，弄一个 Neo4j Aura 免费实例，手工建一个小图。然后映射你自己的领域：从你当前的项目里拿十个实体，逐个归类成 P、O、L、E 或者 O，加上领域标签，开始查询。模型会演进——而这正是全部要点。

For the full scaffolded experience, Create Context Graph supports 27 domains and also lets you describe custom ones in plain English.

想要完整的脚手架体验，Create Context Graph 支持 27 个领域，也允许你用日常语言描述自定义的领域。

参考链接

- [Create Context Graph — GitHub](#) — 本文讨论的那个脚手架工具，POLE+O 本体就在里面
- [Create Context Graph — 主页](#) — 工具官网，支持 27 个领域
- [Introducing Create Context Graph](#) — Neo4j 官方对这个工具的介绍
- [Neo4j Graph Data Modeling — GraphAcademy](#) — 官方免费的图建模基础课
- [原文 \(Medium\)](#) — 这篇文章最初发布的地方